

FSWD-II Paper Solution

Q1 (a) Short Questions (1 mark each)

(i) What is the role of the Socket class in Java networking?

👉 The Socket class in Java represents the **client-side endpoint** of a two-way communication link between a client and a server. It is used to connect to a remote machine and exchange data via input/output streams.

(ii) Define RMI.

👉 RMI (**Remote Method Invocation**) is a Java API that allows an object running in one JVM to invoke methods on an object located in another JVM, enabling distributed applications.

(iii) What is JavaFX primarily used for in Java?

👉 JavaFX is primarily used for building **rich desktop applications** with modern UI features such as 2D/3D graphics, animation, and media playback.

Q1 (b) MCQs / True-False (1 mark each)

1. Which class is used to get the IP address of a machine in Java?
☒ a) InetAddress
2. True or False: Java RMI allows an object running in one JVM to invoke methods on an object in another JVM. Justify.
☒ True – RMI supports remote communication between JVMs by using stubs and skeletons.
3. The URL class in Java can be used to read from a _____.
☒ Web resource (e.g., a webpage or network file).
4. Which of the following is NOT a 2D shape in JavaFX?
☒ b) Sphere (it is a 3D shape).
5. Which method is called to launch a JavaFX application?
☒ launch()
6. What is the purpose of stub and skeleton in RMI?
👉 Stub acts as a client-side proxy forwarding method calls to the server. Skeleton resides on the server side, receiving requests from the stub and invoking actual methods.

7. **JavaFX supports the creation of animation effects.**

 True

Q2 (a) Two Questions of 2 Marks

(i) How does a Java application establish a connection to a server using sockets?

- A client creates a Socket object with the server's IP and port.
- The server listens using ServerSocket.
- Once connected, both use InputStream and OutputStream to exchange data.

(ii) Briefly describe the architecture of JavaFX.

- JavaFX uses a **Scene Graph architecture**.
 - A Stage is the top-level container (window).
 - A Scene holds UI elements.
 - Nodes (controls, shapes, images) form a hierarchical tree structure.
 - It supports FXML for UI design and CSS for styling.
-

Q2 (b) Two Questions of 3 Marks

(i) Steps in developing and running a JavaFX application:

1. Create a class extending Application.
2. Override start(Stage primaryStage).
3. Create UI elements (nodes, scenes).
4. Add scene to stage and call show().
5. Run using launch(args).

(ii) JavaFX Shape and Event Handling Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
```

```

public class ShapeEventDemo extends Application {
    @Override
    public void start(Stage stage) {
        Circle circle = new Circle(100, 100, 50, Color.BLUE);
        circle.setOnMouseClicked(e -> circle.setFill(Color.RED));

        Scene scene = new Scene(new javafx.scene.Group(circle), 300, 200);
        stage.setScene(scene);
        stage.setTitle("JavaFX Event Demo");
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}

```

👉 Here, clicking the circle changes its color.

Q3 Attempt Any TWO (5 Marks each)

(i) TCP vs UDP in Java Sockets:

- **TCP Socket:**
 - Reliable, connection-oriented.
 - Guarantees delivery in correct order.
 - Used for chat apps, file transfer.
- **UDP Socket:**
 - Connectionless, faster, but no reliability.
 - Used for video streaming, gaming, broadcasting.

(ii) Java RMI Client-Server Scenario:

- A bank system where client requests balance check from remote server.
- **Process:**
 - Define remote interface.
 - Implement remote class.
 - Start RMI registry.
 - Client looks up object and calls remote methods.

(iii) Animation in JavaFX:

- JavaFX supports **Timeline**, **PathTransition**, **FadeTransition**.
- Example: Moving a rectangle across the screen.

```
Rectangle rect = new Rectangle(50, 50, Color.BLUE);
```

```
TranslateTransition move = new TranslateTransition(Duration.seconds(3), rect);
```

```
move.setByX(300);
```

```
move.setCycleCount(Timeline.INDEFINITE);
```

```
move.setAutoReverse(true);
```

```
move.play();
```

Q4 (a) Networking Application (5 Marks)

Scenario: Multiple clients request data from a central server.

Architecture:

- **Server:** Uses `ServerSocket` to listen.
- **Clients:** Use `Socket` to connect.
- **Data Transfer:** Server sends responses via `OutputStream`, clients read using `InputStream`.

👉 This ensures multi-client communication.

Q4 (b) JavaFX Program (5 Marks)

Two Shapes with Event Handlers:

```
import javafx.application.Application;
```

```
import javafx.scene.Scene;

import javafx.scene.paint.Color;

import javafx.scene.shape.Circle;

import javafx.scene.shape.Rectangle;

import javafx.scene.layout.HBox;

import javafx.stage.Stage;


public class ShapeColorDemo extends Application {

    @Override

    public void start(Stage stage) {

        Circle circle = new Circle(50, Color.GREEN);

        Rectangle rect = new Rectangle(100, 100, Color.ORANGE);


        circle.setOnMouseClicked(e -> circle.setFill(Color.BLUE));

        rect.setOnMouseClicked(e -> rect.setFill(Color.RED));


        HBox root = new HBox(20, circle, rect);

        Scene scene = new Scene(root, 400, 200);

        stage.setScene(scene);

        stage.setTitle("Shape Event Handling");

        stage.show();

    }

    public static void main(String[] args) {

        launch(args);

    }

}
```